DSA521S - DATA STRUCTURES AND ALGORITHMS 1

GROUP MINI-PROJECT 2026

NUST Service Centre Simulation: Designing and Evaluating Data Structures and Algorithms

PROJECT REPORT

Group Number	Not assigned
Member 1	Japhet Nekwiyu - 225001187
Member 2	Selma Nandago - 225132214
Member 3	Maryna Kashaka - 225097958
Member 4	Penda Haimene - 226141101
Member 5	Mervin Hukununa - 225072068
Submitted by	225001187 - Japhet Nekwiyu

GitHub Repository: https://github.com/JaphetNekwiyu/DSA521S-Group-Mini-Project-2026

Submission deadline: 25 September 2026 at 23:59

Group identification note: No group number was assigned. The group consists of the five registered members listed above. The GitHub repository URL must be inserted in this report and the README after the repository is created.

Contents

1. 1. Project Overview
2. 2. Part A - Data Structure Design
3. 3. Part B - Sorting Algorithm Challenge
4. 4. Part C - Algorithm Experiment
5. 5. Part D - Integrated Service-Centre System
6. 6. Part E - Pseudocode and Algorithm Representation
7. 7. GitHub Repository Link
8. 8. Conclusion

1. Project Overview

The project models a NUST student service centre in Java. Students can arrive for registration assistance, fee enquiries, student card replacement, academic enquiries, document collection and related services. The emphasis is on implementing and explaining the required data structures and algorithms rather than building a graphical user interface.

The integrated program combines a Queue, Singly Linked List, Array processing and Sorting algorithms. The postfix Stack task remains a separate exercise. Built-in Java classes or methods that replace the required data structures or sorting algorithms are not used.

Component	Purpose
Queue	Maintain students in first-in, first-out waiting order.
Singly Linked List	Store student service records and support insertion, deletion, search and traversal.
Stack	Evaluate postfix arithmetic expressions using LIFO behaviour.
Array	Store service times of served students and calculate daily statistics.
Sorting	Sort service times and compare four sorting algorithms experimentally.

2. Part A - Data Structure Design

A1 - Waiting Line: Queue

The waiting line uses an array-based Queue. Students are normally assisted in the same order in which they arrive, so FIFO behaviour directly matches the service-centre requirement.

```
Initial queue after six arrivals:
FRONT                                                    REAR
  |                                                        |
  v                                                        v
[Maria] -> [Tomas] -> [Ndapewa] -> [Simon] -> [Anna] -> [Peter]

After three students are served:
FRONT                      REAR
  |                          |
  v                          v
[Simon] -> [Anna] -> [Peter]
```

Required operation	Implementation behaviour
enqueue(student)	Adds the student at the rear. If the queue was empty, front is set to the first array position.
dequeue()	Removes the front student. The remaining elements are shifted left and rear is decreased.
peek()	Returns the front student without removing the student.
isEmpty()	Checks whether front equals -1.
displayQueue()	Traverses the active array positions from front to rear.

Demonstration: Maria, Tomas, Ndapewa, Simon, Anna and Peter are enqueued. Three dequeue operations serve Maria, Tomas and Ndapewa, leaving Simon at the front followed by Anna and Peter.

A2 - Student Service Records: Singly Linked List

Each linked-list node stores a Student object and a next pointer. The Student object contains the student number, name, service type and estimated service time. The final node points to NULL.

```
Node structure:
[ Student Number | Name | Service Type | Time | Next ] -> next node

Example traversal:
HEAD -> [Maria] -> [Tomas] -> [Ndapewa] -> [Simon] -> NULL
```

Insertion link change

```
Before inserting Anna at position 3:
HEAD -> [Simon] -> [Maria] -> [Tomas] -> [Ndapewa] -> NULL

After:
HEAD -> [Simon] -> [Maria] -> [Anna] -> [Tomas] -> [Ndapewa] -> NULL
```

Deletion link change

```
Before deleting Anna:
HEAD -> [Simon] -> [Maria] -> [Anna] -> [Tomas] -> NULL

After:
HEAD -> [Simon] -> [Maria] -> [Tomas] -> NULL
```

The implementation supports insertion at the beginning, at the end and at a specified position, together with deletion by student number, linear search by student number and traversal for display.

A3 - Postfix Expression Evaluation: Stack

Implementation note: PostfixEvaluator.java uses a manually implemented array-based stack with push(), pop() and peek(). It supports +, -, * and / and traces the required example 5 3 + 2 * = 16.

The postfix task is independent from the integrated service-centre menu. Numbers are pushed onto the stack. When an operator is encountered, the required operands are popped, the operation is performed and the result is pushed back. The required operators are +, -, * and /.

Token	Operation	Stack after operation
5	push 5	[5]
3	push 3	[5, 3]
+	pop 3 and 5; push 8	[8]
2	push 2	[8, 2]
*	pop 2 and 8; push 16	[16]

For the expression 5 3 + 2 *, the final result is 16.

A4 - Daily Statistics: Array

Whenever a student is served in the integrated system, the estimated service time is stored in a manually managed integer array. The program traverses only the populated portion of the array and calculates the required values without max(), min(), sum() or equivalent shortcuts.

- Total students served
- Total service time
- Average service time
- Highest service time
- Lowest service time
- Number of services longer than 10 minutes

```
Example verified output after Maria is served:
Total students served : 1
Total service time     : 12 minutes
Average service time   : 12.00 minutes
Highest service time   : 12 minutes
Lowest service time    : 12 minutes
Services > 10 minutes  : 1
```

A5 - Data-Structure Justification

Structure	Why it is appropriate for the assigned task
Queue	FIFO preserves the real arrival order: the first student to join the waiting line is normally the first student served.
Singly Linked List	Records can be inserted and deleted by changing next links without shifting an entire collection. Traversal and linear search are suitable for the required record operations.
Stack	Postfix evaluation needs the most recently encountered operands first. LIFO provides exactly this access pattern through push and pop.
Array	Service times are homogeneous numeric values. Sequential storage makes traversal for totals, averages, minima, maxima and counts simple and efficient.

3. Part B - Sorting Algorithm Challenge

All four algorithms sort the same service-time array in ascending order: [17, 5, 23, 8, 14, 3, 11, 20, 6, 9]. The final sorted array is [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]. No built-in sorting function is used.

B1 - Selection Sort

Pass	Array after pass
1	[3, 5, 23, 8, 14, 17, 11, 20, 6, 9]
2	[3, 5, 23, 8, 14, 17, 11, 20, 6, 9]
3	[3, 5, 6, 8, 14, 17, 11, 20, 23, 9]

Verified against the supplied algorithm logic: 45 data-value comparisons and 7 swaps are performed for the complete sort.

B2 - Insertion Sort

Pass	Array after pass
1	[5, 17, 23, 8, 14, 3, 11, 20, 6, 9]
2	[5, 17, 23, 8, 14, 3, 11, 20, 6, 9]
3	[5, 8, 17, 23, 14, 3, 11, 20, 6, 9]

Using a comparison counter that counts every data-value comparison, including the comparison that stops each insertion scan, the complete sort performs 32 comparisons and 25 shifts.

B3 - Merge Sort

Base case: a subarray of size 1 (or 0) is already sorted, so recursion stops when left >= right.

```
Divide phase:
[17,5,23,8,14,3,11,20,6,9]
 -> [17,5,23,8,14] | [3,11,20,6,9]
 -> [17,5,23] | [8,14] | [3,11] | [20,6,9]
 -> [17,5] | [23] | [8] | [14] | [3] | [11] | [20] | [6,9]
 -> [17] [5] [23] [8] [14] [3] [11] [20] [6] [9]

Merge phase:
[17]+[5] -> [5,17]
[5,17]+[23] -> [5,17,23]
[8]+[14] -> [8,14]
[5,17,23]+[8,14] -> [5,8,14,17,23]
[3]+[11] -> [3,11]
[6]+[9] -> [6,9]
[20]+[6,9] -> [6,9,20]
[3,11]+[6,9,20] -> [3,6,9,11,20]
Final merge -> [3,5,6,8,9,11,14,17,20,23]
```

B4 - Quick Sort

Pivot-selection rule: the last element of the current subarray is used as the pivot (Lomuto partition scheme).

Stage	Pivot	Left partition	Right partition
1	9	[5, 8, 3, 6]	[17, 23, 14, 11, 20]
2 (left side)	6	[5, 3]	[8]
3 (right side)	20	[17, 14, 11]	[23]

4. Part C - Algorithm Experiment

C1-C2 - Method

The experiment uses input sizes 20, 50, 100 and 500. For each size, one original random array is generated using a fixed Random seed of 42, and every sorting algorithm receives a copy of exactly the same values. Only the sorting call is timed with System.nanoTime(); array generation, copying and printing are outside the measured interval. Comparison counters record only comparisons between data values.

C3 - Random-array results

Algorithm	Input size	Comparisons	Execution time (ns)
Selection Sort	20	190	12,990
Insertion Sort	20	135	9,394
Merge Sort	20	64	13,390
Quick Sort	20	60	7,361
Selection Sort	50	1,225	40,010
Insertion Sort	50	599	27,581
Merge Sort	50	217	31,678
Quick Sort	50	229	14,501
Selection Sort	100	4,950	207,191
Insertion Sort	100	2,806	111,708
Merge Sort	100	540	58,277
Quick Sort	100	650	34,522

Selection Sort	500	124,750	3,638,845
Insertion Sort	500	62,525	2,328,541
Merge Sort	500	3,860	374,291
Quick Sort	500	5,139	289,073

Timing values are from one verified run of the integrated Java experiment and may vary between machines or runs because System.nanoTime() reflects the local runtime environment.

C4 - Almost-sorted 100-element test

The same original 100-element random array is copied and sorted in ascending order. Exactly five non-overlapping pairs of neighbouring values are then swapped. The identical almost-sorted array is copied to each algorithm for the comparison.

Algorithm	Comparisons	Execution time (ns)
Selection Sort	4,950	172,448
Insertion Sort	103	5,889
Merge Sort	357	56,855
Quick Sort	4,922	328,142

C5 - Analysis of Results

1. Which algorithm performed the fewest comparisons? Quick Sort used the fewest comparisons at n=20 (60 versus Merge Sort's 64). From n=50 onward, Merge Sort used the fewest comparisons, and it had the strongest overall comparison growth as the inputs became larger.

2. Which algorithm performed the most comparisons? Selection Sort performed the most comparisons at every tested random input size: 190, 1,225, 4,950 and 124,750.

3. Which algorithm performed best on the almost-sorted array? Insertion Sort performed best. It required only 103 comparisons and completed in 5,889 ns in this run, far below Selection Sort and Quick Sort on the almost-sorted input.

4. Which algorithm appeared to scale best? The O(n log n) algorithms scaled much better than the O(n^2) algorithms. Merge Sort showed the most consistent low comparison growth, while Quick Sort recorded the lowest execution time at n=500 in this run.

5. Did the experimental results generally agree with theoretical time complexity? Yes. Selection and Insertion showed quadratic growth on random data. Merge Sort remained close to n log n. Quick Sort was efficient on the random arrays, but its last-element pivot rule performed poorly on the almost-sorted input, which demonstrates its O(n^2) worst-case behaviour.

6. Is execution time alone enough to decide which algorithm is better? No. Timing depends on hardware, JVM warm-up, compiler/JIT optimisation and data distribution. Comparison counts, input characteristics, memory use, worst-case behaviour, stability and theoretical complexity should also be considered.

5. Part D - Integrated Service-Centre System

The Queue, Singly Linked List, Array and Sorting components operate in one menu-driven Java application named CampusServiceCentre. The postfix Stack exercise remains separate, as required.

Menu and component mapping

Menu option	Operation used
1. Add student to waiting queue	Queue - enqueue
2. Serve next student	Queue - dequeue; then record service time and completed record
3. Display waiting students	Queue - traversal/display
4. Add student service record	Singly Linked List - insertion
5. Display student service records	Singly Linked List - traversal/display
6. Search for student record	Singly Linked List - search
7. Remove student record	Singly Linked List - deletion
8. Display daily statistics	Array processing
9. Sort service times	Selection / Insertion / Merge / Quick Sort

10. Run sorting experiment	Part C experiment
11. Exit	Terminates program

Integration workflow

```
Student details entered
        |
        v
 Queue.enqueue(student)
        |
        v
 Waiting line (FIFO)
        |
        v
 Queue.dequeue() when served
       / \
      v   v
Linked-list record    Service-time array
      |                      |
      v                      v
Search/display/delete   Daily statistics
                             |
                             v
                      Sorting algorithms
```

A key integration point is menu option 2. When a student is dequeued, the student is added to the linked-list service records and the student's estimated service time is appended to the statistics array. This allows the daily statistics and service-time sorting options to operate on students who have actually been served.

Working-system evidence

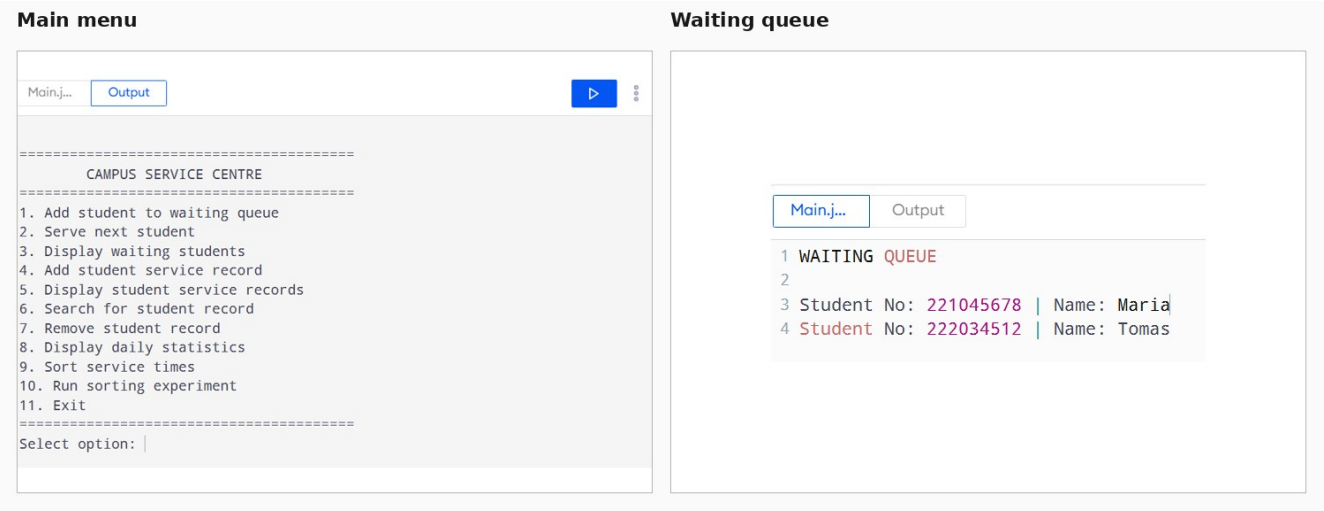


Figure 1. Main menu and waiting queue from the working integrated program.

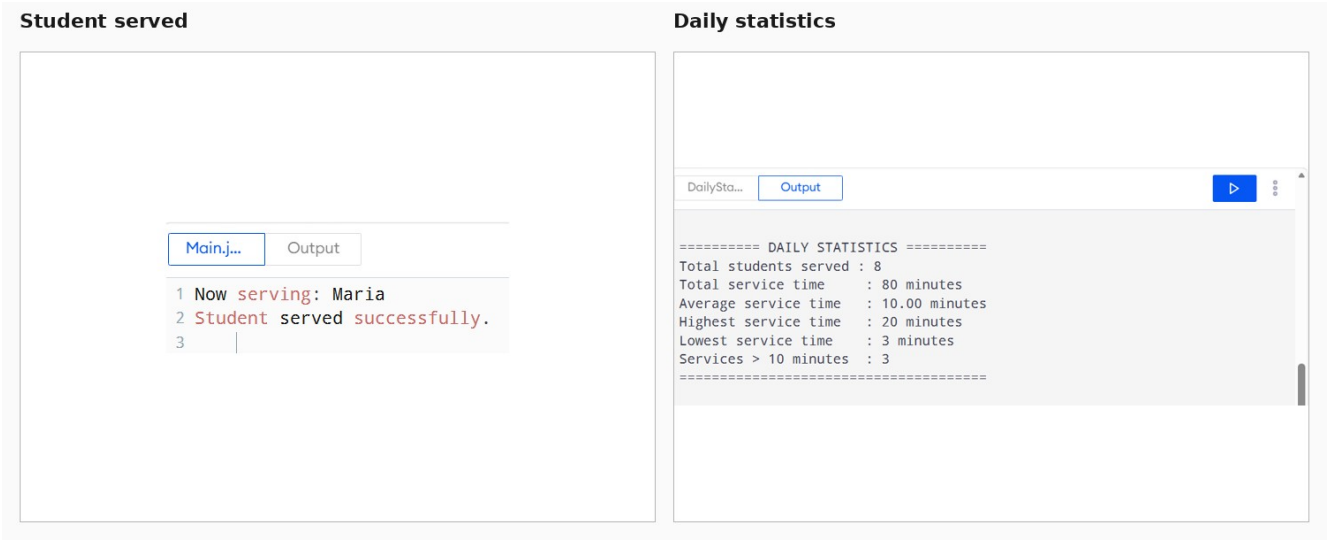


Figure 2. A student is served and the updated daily statistics are displayed.

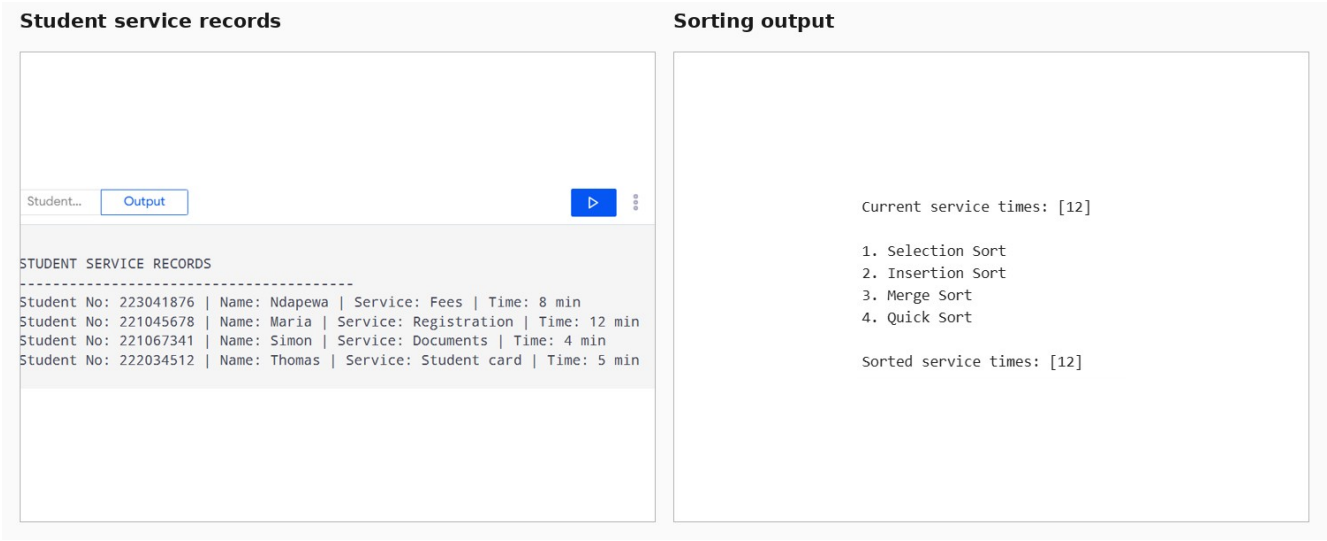


Figure 3. Student service records and service-time sorting output from the integrated program.

6. Part E - Pseudocode and Algorithm Representation

The following pseudocode reflects the logic used by the submitted Queue, Linked List, Array integration and sorting implementations. The stack pseudocode corresponds to the separate PostfixEvaluator.java implementation included with the project source.

Queue pseudocode

```
ENQUEUE(student)
    IF rear == queue.length - 1
        DISPLAY "Queue is full"
        RETURN
    END IF
    IF front == -1
        front = 0
    END IF
    rear = rear + 1
    queue[rear] = student
END ENQUEUE
```

```
DEQUEUE()
    IF front == -1
        DISPLAY "Queue is empty"
        RETURN NULL
    END IF
    student = queue[front]
    FOR i = front TO rear - 1
        queue[i] = queue[i + 1]
    END FOR
    queue[rear] = NULL
    rear = rear - 1
    IF rear == -1
        front = -1
    END IF
    RETURN student
END DEQUEUE
```

Stack pseudocode

```
PUSH(value)
    IF top == stack.length - 1
        DISPLAY "Stack overflow"
        RETURN
    END IF
    top = top + 1
    stack[top] = value
END PUSH

POP()
    IF top == -1
        DISPLAY "Stack underflow"
        RETURN ERROR
    END IF
    value = stack[top]
    top = top - 1
    RETURN value
END POP
```

```
EVALUATE_POSTFIX(expression)
    FOR each token in expression
        IF token is a number
            PUSH(token)
        ELSE IF token is an operator
            right = POP()
            left = POP()
            result = APPLY operator to left and right
            PUSH(result)
        END IF
    END FOR
    RETURN POP()
END EVALUATE_POSTFIX
```

Singly Linked List pseudocode

```
INSERT_NODE(student, position)
    CREATE newNode containing student
    IF position == 1
        newNode.next = head
        head = newNode
        RETURN
    END IF
    current = head
    FOR i = 1 TO position - 2
        IF current == NULL
            DISPLAY "Invalid position"
            RETURN
        END IF
        current = current.next
    END FOR
    IF current == NULL
        DISPLAY "Invalid position"
        RETURN
    END IF
    newNode.next = current.next
    current.next = newNode
END INSERT_NODE

DELETE_NODE(studentNumber)
    IF head == NULL
        DISPLAY "List is empty"
        RETURN
    END IF
    IF head.data.studentNumber == studentNumber
        head = head.next
        RETURN
    END IF
    current = head
    WHILE current.next != NULL
        IF current.next.data.studentNumber == studentNumber
            current.next = current.next.next
            RETURN
        END IF
        current = current.next
    END WHILE
    DISPLAY "Student not found"
END DELETE_NODE

SEARCH_NODE(studentNumber)
    current = head
    WHILE current != NULL
        IF current.data.studentNumber == studentNumber
            RETURN current.data
        END IF
        current = current.next
    END WHILE
    RETURN NOT FOUND
END SEARCH_NODE

TRAVERSE_LIST()
    current = head
    WHILE current != NULL
        DISPLAY current.data
        current = current.next
    END WHILE
END TRAVERSE_LIST
```

Sorting pseudocode

```
SELECTION_SORT(a)
    FOR i = 0 TO length(a) - 2
        minIndex = i
        FOR j = i + 1 TO length(a) - 1
            IF a[j] < a[minIndex]
                minIndex = j
            END IF
        END FOR
        IF minIndex != i
            SWAP a[i] WITH a[minIndex]
        END IF
    END FOR
END SELECTION_SORT
```

```
INSERTION_SORT(a)
    FOR i = 1 TO length(a) - 1
        key = a[i]
        j = i - 1
        WHILE j >= 0 AND a[j] > key
            a[j + 1] = a[j]
            j = j - 1
        END WHILE
        a[j + 1] = key
    END FOR
END INSERTION_SORT
```

```
MERGE_SORT(a, left, right)
    IF left < right
        mid = (left + right) / 2
        MERGE_SORT(a, left, mid)
        MERGE_SORT(a, mid + 1, right)
        MERGE(a, left, mid, right)
    END IF
END MERGE_SORT

MERGE compares the first unmerged values in temporary left and right arrays, copies the smaller
value back to a, and then copies any remaining values.
```

```
QUICK_SORT(a, low, high)
    IF low < high
        pivotIndex = PARTITION(a, low, high)
        QUICK_SORT(a, low, pivotIndex - 1)
        QUICK_SORT(a, pivotIndex + 1, high)
    END IF
END QUICK_SORT

PARTITION(a, low, high)
    pivot = a[high]
    i = low - 1
    FOR j = low TO high - 1
        IF a[j] <= pivot
            i = i + 1
            SWAP a[i] WITH a[j]
        END IF
    END FOR
    SWAP a[i + 1] WITH a[high]
    RETURN i + 1
END PARTITION
```

7. GitHub Repository Link

GitHub Repository Link: [ADD AFTER REPOSITORY IS CREATED]. Before the final eLearning submission, place the same repository URL here and in README.md.

https://github.com/JaphetNekwiyu/DSA521S-Group-Mini-Project-2026

8. Conclusion

The project demonstrates the selection, implementation and analysis of core data structures and algorithms in a realistic service-centre scenario. The Queue maintains arrival order, the Singly Linked List manages service records, the Array supports daily statistics, and the four sorting algorithms provide both functional sorting and experimental performance comparisons. The integrated menu demonstrates that these components can operate together in one Java program while the postfix Stack remains an independent algorithmic exercise.

The experimental results show the practical difference between quadratic and O(n log n) sorting behaviour and also demonstrate the influence of input order and pivot choice. Every group member should be able to explain the data structures, pseudocode, traces and implementation decisions contained in this report.